

React Virtual DOM & Reconciliation - Complete Study Notes

state variable, array changes, clock, keys list

1. Foundation: The Real Problem {#foundation}

What is a User Interface?

At the most fundamental level:

- **Data (state)** that changes over time
- **Visual representation** of that data on screen

The Core Challenge

When data changes, how do we update the visual representation efficiently?

Naive Approach 1: Destroy and Recreate Everything

```
let count = 0;

function render() {
  const container = document.getElementById('app');
  container.innerHTML = ''; // Destroy everything
  container.innerHTML = `
    <div>
      <h1>Count: ${count}</h1>
      <button onclick="increment()">+</button>
    </div>
  `;
}
```

```
`; // Recreate from scratch  
}
```

Problems:

- ✗ Wasteful: Destroys and recreates entire DOM tree
- ✗ Loses state: Input values, focus, scroll position
- ✗ Event listeners lost
- ✗ Poor performance: For 1000 elements, recreates all 1000 even if only 1 changed

Naive Approach 2: Manual Surgical Updates

```
let count = 0;  
  
function render() {  
  const countElement = document.querySelector('.count-value');  
  countElement.textContent = count; // Update only what changed  
}
```

Problems:

- ✗ Manual tracking: YOU must remember which elements to update
- ✗ Doesn't scale: Complex UIs = many manual updates
- ✗ Easy to forget updates → bugs
- ✗ Hard to maintain

The Key Insight: Declarative UI

Imperative (HOW to update):

```
if (count changed) {  
  find element with class "count-value"
```

```
    update its text to new count
  }
```

Declarative (WHAT it should be):

```
function UI(count) {
  return `<div><h1>Count: ${count}</h1></div>`;
}
```

We want:

- ☒ Write declarative code (describe WHAT)
- ☒ Get efficient updates (only change WHAT changed)

2. Virtual DOM Concept {#virtual-dom}

The Solution: A Middleman

```
Developer writes WHAT UI should be
      ↓
[Virtual DOM] ← Figures out HOW to update efficiently
      ↓
Real DOM updated
```

What is Virtual DOM?

Virtual DOM = Lightweight JavaScript object representation of your UI

Instead of immediately creating expensive DOM elements:

```
// Expensive - involves browser rendering engine
const realDOM = document.createElement('div');
realDOM.className = 'container';

// Cheap - just memory allocation
const virtualDOM = {
```

```
type: 'div',
props: { className: 'container' },
children: []
};
```

Key Insight: Creating JavaScript objects is ~100x faster than creating DOM elements because:

- No browser rendering engine involvement
- No style calculation
- No layout computation
- Just memory allocation

Example: Todo App

```
const state = {
  todos: [
    { id: 1, text: "Learn React", done: false },
    { id: 2, text: "Build app", done: false }
  ]
};

// Virtual DOM representation
function TodoApp(state) {
  return {
    type: 'div',
    props: { className: 'app' },
    children: [
      {
        type: 'h1',
        props: {},
        children: ['Todo List']
      },
      {
        type: 'ul',
```

```

    props: {},
    children: state.todos.map(todo => ({
      type: 'li',
      props: { className: todo.done ? 'done' : '' },
      children: [todo.text]
    }))
  }
]
};
}

```

How Virtual DOM Works

Step 1: Initial Render

1. Create Virtual DOM (JavaScript objects)
2. Convert Virtual DOM to Real DOM (only done once)
3. Store Virtual DOM for future comparisons

Step 2: State Changes

1. Create NEW Virtual DOM with updated state
2. Now we have TWO Virtual DOMs in memory:
 - Old Virtual DOM (current UI)
 - New Virtual DOM (desired UI)

Step 3: Diffing/Reconciliation

1. Compare old and new Virtual DOMs
2. Find minimal differences
3. This happens entirely in JavaScript (fast!)

Step 4: Commit

1. Apply only the changes to Real DOM
2. Only minimal browser work needed

Why This is Efficient

Without Virtual DOM:

State change → Destroy 100 DOM elements → Create 100 DOM elements
→ Browser: Recalculate styles × 100, Reflow × 100, Paint × 100

With Virtual DOM:

State change → Create new Virtual DOM (fast JS objects)
→ Diff old vs new (fast JS comparison)
→ Apply ONLY changes to Real DOM (minimal browser work)

3. Reconciliation Algorithm {#reconciliation}

The Problem: Tree Comparison is $O(n^3)$

Traditional tree comparison algorithms:

```
function naiveTreeDiff(tree1, tree2) {  
  for (let node1 of tree1) {           // O(n)  
    for (let node2 of tree2) {         // O(n)  
      calculateEditDistance(node1, node2); // O(n) - compare  
      s entire subtrees  
    }  
  }  
}  
// Total:  $O(n^3)$ 
```

Why $O(n^3)$?

- Outer loop: n nodes
- Inner loop: n nodes
- Edit distance: Must traverse both subtrees ($O(n)$ work)

For 1000 nodes = 1 billion operations!

React's Solution: $O(n)$ with Heuristics

React makes two assumptions:

Assumption 1: Different Types = Different Trees

```
// Old
<div><Counter /></div>

// New
<div><Timer /></div>

// React: Different types (Counter vs Timer)
// → Don't compare subtrees, just replace entirely
```

Trade-off: Sometimes recreates more than necessary, but simplicity and speed win.

Assumption 2: Keys Identify Stable Elements

Keys help React know "this is the same element, just moved" (covered in detail below).

The Reconciliation Process

```
function reconcile(oldNode, newNode) {
  // Rule 1: Different types → Replace entirely
  if (oldNode.type !== newNode.type) {
    return { type: 'REPLACE', oldNode, newNode };
  }

  // Rule 2: Same type → Compare props and children
  const changes = [];

  // Compare props
  const propChanges = compareProps(oldNode.props, newNode.pro
```

```

ps);
  if (propChanges.length > 0) {
    changes.push({ type: 'UPDATE_PROPS', changes: propChanges
  });
}

// Compare children recursively
const childChanges = reconcileChildren(oldNode.children, ne
wNode.children);
changes.push(...childChanges);

return changes;
}

```

Example: Step-by-Step Reconciliation

```

function App({ user }) {
  return (
    <div className="app">
      <h1>Hello {user.name}</h1>
      <p>Age: {user.age}</p>
    </div>
  );
}

// Old: { name: 'John', age: 25 }
// New: { name: 'Jane', age: 25 }

```

Reconciliation steps:

1. **Compare** `<div>`: Same type, same props → Continue to children
2. **Compare** `<h1>`: Same type → Compare children
 - Text "Hello John" vs "Hello Jane" → Different! Mark for update
3. **Compare** `<p>`: Same type → Compare children

- Text "Age: 25" vs "Age: 25" → Same, skip

Result: Only one text node update needed!

```
changes = [{
  type: 'UPDATE_TEXT',
  path: ['div', 'h1', 1],
  oldText: 'John',
  newText: 'Jane'
}];

// Apply to DOM:
document.querySelector('.app h1').childNodes[1].nodeValue =
'Jane';
```

Comparing Props

```
function compareProps(oldProps, newProps) {
  const changes = [];

  // Check changed or removed props
  for (let key in oldProps) {
    if (!(key in newProps)) {
      changes.push({ key, oldValue: oldProps[key], newValue:
null });
    } else if (oldProps[key] !== newProps[key]) {
      changes.push({ key, oldValue: oldProps[key], newValue:
newProps[key] });
    }
  }

  // Check added props
  for (let key in newProps) {
    if (!(key in oldProps)) {
      changes.push({ key, oldValue: null, newValue: newProps
[key] });
    }
  }
}
```

```

    }
  }

  return changes;
}

```

Important: React uses **reference equality (===)** to compare props!

```

// These are different references
const obj1 = { name: 'John' };
const obj2 = { name: 'John' };
console.log(obj1 === obj2); // false

// This causes re-renders:
<Child data={{ name: 'John' }} /> // NEW object every render!

```

4. Keys and List Rendering {#keys}

The Problem: Position-Based Comparison

Without keys:

```

// Old
['A', 'B', 'C']

// New (added 'X' at start)
['X', 'A', 'B', 'C']

// React compares by position:
Position 0: 'A' vs 'X' → Different! UPDATE
Position 1: 'B' vs 'A' → Different! UPDATE
Position 2: 'C' vs 'B' → Different! UPDATE
Position 3: nothing vs 'C' → CREATE

```

```
// 4 DOM operations for adding 1 item!
```

The Solution: Keys

```
// Old
<li key="a">A</li>
<li key="b">B</li>
<li key="c">C</li>

// New
<li key="x">X</li>
<li key="a">A</li>
<li key="b">B</li>
<li key="c">C</li>

// React compares by KEY:
key='x': not in old → CREATE
key='a': in old → REUSE (just moved)
key='b': in old → REUSE (just moved)
key='c': in old → REUSE (just moved)

// Only 1 DOM creation!
```

How Keys Work (Algorithm)

```
function reconcileWithKeys(oldChildren, newChildren) {
  // Step 1: Build map of old children by key - O(n)
  const oldMap = new Map();
  oldChildren.forEach(child => {
    if (child.key !== null) {
      oldMap.set(child.key, child);
    }
  });
}
```

```

// Step 2: Process new children - O(n)
newChildren.forEach(newChild => {
  const oldChild = oldMap.get(newChild.key); // O(1) lookup!

  if (oldChild) {
    // Item exists, reuse it
    reconcile(oldChild, newChild);
  } else {
    // New item, create it
    createNewElement(newChild);
  }
});
}

// Total: O(n) time complexity

```

Why Index as Key is Bad

```

// BAD
{todos.map((todo, index) => (
  <li key={index}>{todo.text}</li>
))}

```

Problem:

```

// Old
<li key={0}>A</li>
<li key={1}>B</li>
<li key={2}>C</li>

// New (added 'X' at start)
<li key={0}>X</li> // key=0 now points to different item!
<li key={1}>A</li> // key=1 now points to different item!
<li key={2}>B</li>

```

```
<li key={3}>C</li>
```

```
// React thinks items at key=0, 1, 2 all changed!  
// Back to 4 DOM operations
```

State Loss Example

```
function Counter({ name }) {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <span>{name}: {count}</span>  
      <button onClick={() => setCount(count + 1)}>+</button>  
    </div>  
  );  
}  
  
// With index as key  
{items.map((item, index) => (  
  <Counter key={index} name={item} />  
))}
```

User interaction:

1. User increments "Counter A" to 5
2. You add "Counter X" at the beginning
3. **Bug:** "Counter X" now shows count=5 (got Counter A's old state!)

Why? State is attached to the key, not the data. When you use index as key and insert an item, the indices shift, causing state to be attached to wrong items.

Rules for Keys

1.  **Unique** among siblings
2.  **Stable** (same item = same key across renders)

3. **Predictable** (not random)

```
//  BAD
{items.map((item, index) => <li key={index}>{item}</li>)}
{items.map(item => <li key={Math.random()}>{item}</li>)}

//  GOOD
{items.map(item => <li key={item.id}>{item}</li>)}
```

5. React Fiber Architecture {#fiber}

What is Fiber?

Fiber is React's implementation of Virtual DOM (since React 16, 2017)

Before Fiber: **Stack Reconciler** (couldn't be interrupted)

After Fiber: **Fiber Reconciler** (can pause/resume work)

The Problem Fiber Solves

Without Fiber (Stack Reconciler):

```
User clicks button
  ↓
React starts reconciliation (BLOCKING)
  ↓
Processing 1000 components... (takes 200ms)
  ↓
[User tries to type - FROZEN!]
  ↓
After 200ms: reconciliation done, DOM updates
  ↓
Now user can type
```

UI freezes = bad user experience!

With Fiber:

```
User clicks button
↓
React starts reconciliation
↓
Processing components...
↓
[User tries to type - HIGH PRIORITY!]
↓
React PAUSES reconciliation
↓
Handle user typing immediately
↓
Resume reconciliation later
↓
Commit changes when done
```

Fiber Node Structure

Instead of a simple tree, Fiber uses a **linked list** structure:

```
{
  // Identity
  type: 'div' | Function,
  key: string | null,

  // Linked list pointers
  child: Fiber | null,    // First child
  sibling: Fiber | null,  // Next sibling
  return: Fiber | null,  // Parent

  // Props and State
  memoizedProps: any,    // Props from last render
  pendingProps: any,     // Props for current render
  memoizedState: any,    // State (hooks)
```

```

// DOM reference
stateNode: DOMNode | null, // Actual DOM element

// Work tracking
alternate: Fiber | null,    // Previous version
effectTag: number,          // What to do (update, delete, e
tc.)
}

```

Why Linked List?

Can pause and resume work:

```

function workLoop(deadline) {
  let shouldYield = false;

  while (nextUnitOfWork && !shouldYield) {
    // Do small chunk of work
    nextUnitOfWork = performUnitOfWork(nextUnitOfWork);

    // Check: Do we have time left?
    shouldYield = deadline.timeRemaining() < 1;
  }

  if (nextUnitOfWork) {
    // More work to do, continue later
    requestIdleCallback(workLoop);
  } else {
    // Done, commit changes
    commitRoot();
  }
}

```

Benefits of Fiber

1. **Time Slicing:** Break work into chunks

2. **Priority Scheduling:** Handle urgent updates first
 3. **Pausable/Resumable:** Can interrupt low-priority work
 4. **Concurrent Rendering:** Work on multiple versions of UI
-

6. Two-Phase Rendering {#two-phase}

React splits rendering into two phases:

Phase 1: Render Phase (Reconciliation)

What happens:

- Call component functions
- Create new Fiber tree
- Compare with old Fiber tree
- Build list of changes (effects)

Characteristics:

-  Can be interrupted/paused
-  Can be resumed later
-  Can be abandoned if higher priority update comes
-  Pure computation (no side effects)
-  Doesn't touch DOM

Example:

```
function App() {  
  const [count, setCount] = useState(0);  
  
  // This runs in Render Phase  
  const expensiveValue = heavyComputation(count);  
  
  return <div>{expensiveValue}</div>;  
}
```

```
// When setCount is called:  
// 1. React calls App() - can be paused here  
// 2. Runs heavyComputation - can be paused here  
// 3. Creates new Fiber tree - can be paused here  
// 4. Compares with old tree - can be paused here
```

Phase 2: Commit Phase

What happens:

- Apply all DOM updates
- Call lifecycle methods (useEffect, useLayoutEffect)
- Update refs

Characteristics:

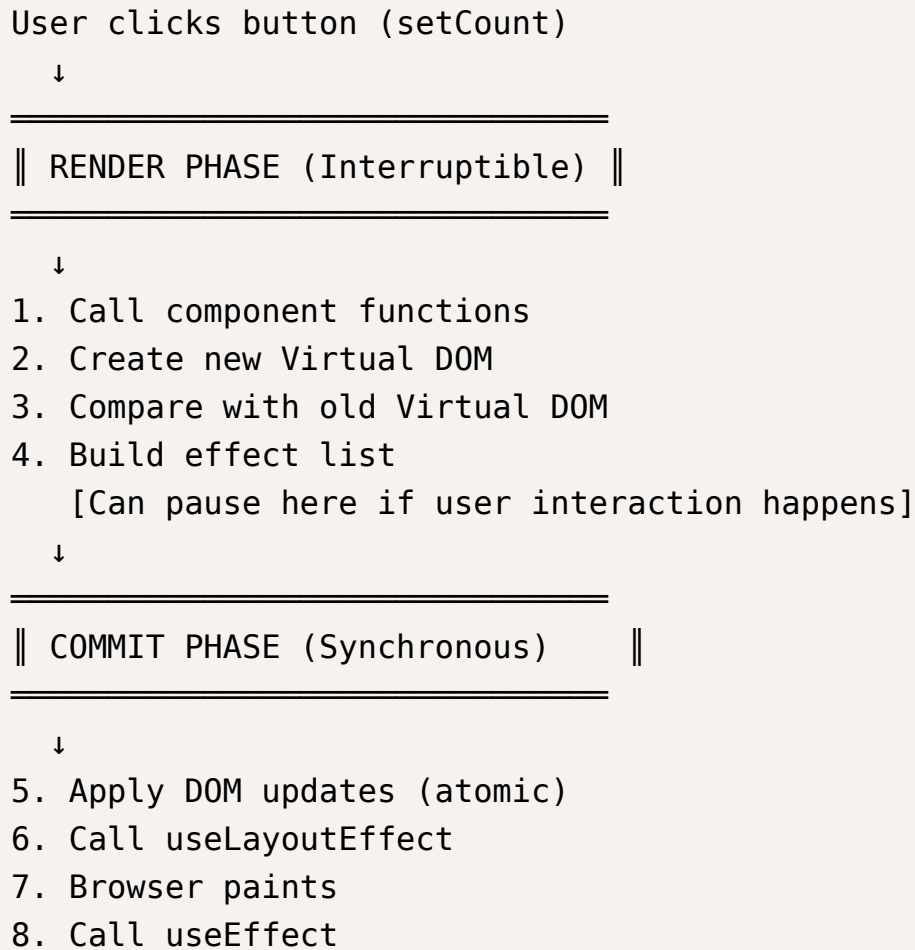
- ❌ Cannot be interrupted (synchronous)
- ❌ Cannot be paused
- ✅ Side effects happen here
- ✅ Mutates the DOM

Why can't it be interrupted?

If Commit Phase could pause, UI would be inconsistent:

```
// If paused mid-commit:  
// Step 1: Update Header balance: $500 → $400  
// [PAUSE - higher priority update]  
// Step 2: Update Footer balance: $500 → $400 [NOT EXECUTED Y  
ET]  
  
// Screen shows:  
// Header: $400  
// Footer: $500 ← INCONSISTENT!
```

Visual Flow



Priority Scheduling

```
function SearchApp() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  function handleInput(e) {
    // High priority - user typing
    setQuery(e.target.value);

    // Low priority - background update
    startTransition(() => {
```

```

        setResults(expensiveFilter(data, e.target.value));
    });
}

return (
    <>
        <input value={query} onChange={handleInput} />
        <Results data={results} />
    </>
);
}

```

What happens:

```

User types 'a'
↓
[High Priority] Update input to 'a'
  [Commit Phase] Input shows 'a' immediately
↓
[Low Priority] Start filtering for 'a'
  [Render Phase] Processing...
↓
User types 'b' (before filtering finishes)
↓
❌ Abandon filtering for 'a'
↓
[High Priority] Update input to 'ab'
  [Commit Phase] Input shows 'ab' immediately
↓
[Low Priority] Filter for 'ab'

```

7. State Updates & Batching {#batching}

Automatic Batching

When multiple state updates happen in the same event, React **batches** them into a single re-render:

```
function App() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState('John');
  const [active, setActive] = useState(false);

  console.log('Component rendered');

  function handleClick() {
    setCount(count + 1); // Queued
    setName('Jane');     // Queued
    setActive(true);     // Queued
    // All 3 updates batched together!
  }

  // Result: console.log appears only ONCE
}
```

Without batching:

```
setCount(1) → Render 1
setName('Jane') → Render 2
setActive(true) → Render 3
```

Total: 3 renders, 3 Virtual DOM creations, 3 reconciliations

With batching (React's actual behavior):

```
setCount(1) → Queue
setName('Jane') → Queue
setActive(true) → Queue
↓
(Event handler finishes)
↓
```

Merge all updates: { count: 1, name: 'Jane', active: true }

↓

Single render with all new values

↓

Create ONE Virtual DOM, ONE reconciliation

Total: 1 render

How Batching Works

```
// Step 1: Queue updates
function handleClick() {
  setCount(1);    // Queue: [{ key: 'count', value: 1 }]
  setName('Jane'); // Queue: [{ key: 'count', value: 1 }, { key: 'name', value: 'Jane' }]
  setActive(true); // Queue: [..., { key: 'active', value: true }]
}
```

```
// Step 2: After event handler completes
// React merges all updates
newState = { count: 1, name: 'Jane', active: true }
```

```
// Step 3: Call component ONCE with merged state
App() // count=1, name='Jane', active=true
```

Batching in React 18+

In React 18, batching works everywhere:

```
// ✅ Event handlers (always batched)
function handleClick() {
  setCount(1);
  setName('Jane');
}
```

```
// ✅ Timeouts (batched in React 18+)
setTimeout(() => {
  setCount(1);
  setName('Jane');
}, 1000);

// ✅ Promises (batched in React 18+)
fetch('/api').then(() => {
  setCount(1);
  setName('Jane');
});

// ✅ Native events (batched in React 18+)
element.addEventListener('click', () => {
  setCount(1);
  setName('Jane');
});
```

State Change Optimization

React skips re-renders when state doesn't change:

```
const [count, setCount] = useState(5);

setCount(5); // 5 === 5 → No re-render
setCount(5); // 5 === 5 → No re-render
setCount(10); // 5 !== 10 → Re-render happens
```

For objects/arrays (reference comparison):

```
const [user, setUser] = useState({ name: 'John', age: 25 });

// ❌ Won't trigger re-render (same reference)
user.age = 26;
setUser(user);
```

```
// ✅ Will trigger re-render (new reference)
setUser({ ...user, age: 26 });
```

8. How React Tracks DOM Elements {#tracking}

Does React Use IDs?

No! React doesn't assign IDs like `id="react-1"`. Instead, it uses **direct object references**.

The stateNode Property

```
// Fiber node structure
{
  type: 'div',
  stateNode: ??? // ← This holds a direct reference to the DOM element!
}
```

How It Works

Initial Render:

```
// React creates actual DOM elements
const divElement = document.createElement('div');
divElement.className = 'container';

const h1Element = document.createElement('h1');
h1Element.textContent = 'Hello';

divElement.appendChild(h1Element);
document.body.appendChild(divElement);

// React stores direct references
```



```

const divFiber = {
  type: 'div',
  stateNode: divElement, // ← Direct JavaScript reference!
};

const h1Fiber = {
  type: 'h1',
  stateNode: h1Element, // ← Direct JavaScript reference!
};

```

Update:

```

// When state changes, React creates new Virtual DOM
const newH1Fiber = {
  type: 'h1',
  children: ['Hello World'], // Changed
  stateNode: null
};

// Reconciliation
if (oldH1Fiber.type === newH1Fiber.type) {
  // Same type! Reuse the existing DOM element
  newH1Fiber.stateNode = oldH1Fiber.stateNode; // Copy reference

  // Update the existing DOM element directly
  newH1Fiber.stateNode.textContent = 'Hello World';
}

```

No ID Lookup Needed!

Traditional approach (jQuery):

```

// Must search DOM
const element = document.getElementById('my-element');

```

```
element.textContent = 'new text';  
// O(n) search time
```

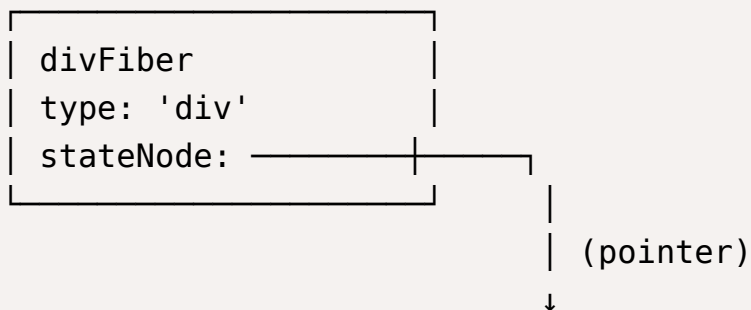
React's approach:

```
// Already have reference!  
const element = fiber.stateNode;  
element.textContent = 'new text';  
// O(1) instant access
```

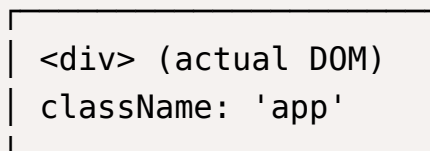
Visual Representation

JavaScript Memory:

Fiber Tree:



Real DOM:



Complete Update Flow

```
// Initial: Create and store reference  
const pElement = document.createElement('p');  
pElement.textContent = 'Count: 0';  
pFiber.stateNode = pElement;
```

```
// Update: Use stored reference
function updateCounter(fiber, newValue) {
  const element = fiber.stateNode; // O(1) access
  element.textContent = `Count: ${newValue}`;
  // No searching, no IDs!
}
```

9. React 19 Updates {#react-19}

Does React 19 Still Use Fiber?

Yes! React 19 still uses:

-  Fiber Architecture
-  Reconciliation Algorithm ($O(n)$)
-  Virtual DOM
-  Two-Phase Rendering
-  Keys for Lists
-  Automatic Batching

What's New: React Compiler

The React Compiler is an **opt-in** build-time optimization tool.

What it does:

- Automatically adds memoization at build time
- Reduces need for manual `useMemo`, `useCallback`, `memo`
- Works **on top of** Fiber, doesn't replace it

Before Compiler (Manual):

```
function TodoList({ todos }) {
  const filtered = useMemo(
    () => todos.filter(t => t.active),
    [todos]
  );
}
```

```

    );

    return <ul>{filtered.map(todo => <Item key={todo.id} todo=
{todo} />)}</ul>;
}

const Item = memo(({ todo }) => {
  return <li>{todo.text}</li>;
});

```

With Compiler (Automatic):

```

function TodoList({ todos }) {
  // Write simple code
  const filtered = todos.filter(t => t.active);

  return <ul>{filtered.map(todo => <Item key={todo.id} todo=
{todo} />)}</ul>;
}

function Item({ todo }) {
  return <li>{todo.text}</li>;
}

// Compiler automatically optimizes at build time!

```

How to Enable React Compiler

```
npm install babel-plugin-react-compiler
```

Vite config:

```

// vite.config.js
export default defineConfig({
  plugins: [

```

```

    react({
      babel: {
        plugins: [['babel-plugin-react-compiler']]
      }
    })
  ]
});

```

Next.js config:

```

// next.config.js
module.exports = {
  experimental: {
    reactCompiler: true
  }
};

```

Current Best Practice (2024)

For most projects:

- ☒ Continue using `useMemo`, `useCallback`, `memo` when needed
- ☒ React Compiler is opt-in, not default
- ☒ Learn manual optimization to understand performance

The compiler:

- Reduces boilerplate
- But doesn't change how React works fundamentally
- Still uses Fiber and reconciliation under the hood

Key Takeaways

Core Principles

1. **Virtual DOM = JavaScript representation of UI**

- Cheap to create and compare
 - Enables declarative code
2. **Reconciliation = Comparing old and new Virtual DOM**
 - $O(n)$ algorithm using heuristics
 - Finds minimal changes needed
 3. **Keys = Identity for list items**
 - Help React know which elements are the same
 - Essential for performance and correctness
 4. **Fiber = Current implementation of Virtual DOM**
 - Allows work to be paused/resumed
 - Enables priority scheduling
 5. **Two Phases = Render (interruptible) + Commit (synchronous)**
 - Render: Figure out what changed
 - Commit: Apply changes atomically
 6. **Batching = Combine multiple state updates**
 - Single re-render for multiple `setState` calls
 - Major performance optimization
 7. **Direct References = How React tracks DOM**
 - No IDs needed
 - $O(1)$ access via `stateNode` pointer

Common Pitfalls

- ✗ Using index as key in dynamic lists
- ✗ Mutating objects/arrays directly
- ✗ Creating new objects/functions in render
- ✗ Forgetting keys when conditional rendering changes structure

